Household Electricity Consumption Prediction Using Machine Learning

(Daily & Monthly Electricity Consumption Prediction using Regression Algorithms)

Problem Statement:

Residential electricity consumption represents a substantial proportion of global energy demand and is subject to significant financial and environmental implications. In contemporary households, daily electricity usage fluctuates dynamically based on household occupancy, seasonal weather patterns, appliance ownership, and intensive cooling loads such as air conditioning. Without data-driven forecasting mechanisms, consumers frequently face unexpectedly high electricity bills, while municipal energy grids struggle with demand spikes and inefficient load scheduling.

Manual calculations or static heuristics are inherently incapable of capturing the multi-dimensional and non-linear interactions between demographic variables, ambient outdoor temperatures, appliance counts, and baseline consumption history. There is, therefore, a pressing requirement for an automated, fast, transparent, and accessible system capable of accurately predicting daily electricity consumption in kilowatt-hours per day (kWh/day) and computing estimated monthly usage (kWh/month) from readily observable household inputs.

This project designs, trains, and implements an end-to-end web-based Household Electricity Consumption Prediction system using supervised machine learning regression algorithms. The system performs data preprocessing, exploratory data analysis (EDA), multi-model training and comparison (Linear Regression, Decision Tree Regressor, Random Forest Regressor), hyperparameter configuration, model persistence via Joblib, and live inference through an interactive Flask web interface. The trained models are systematically evaluated using standard regression metrics including Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and the Coefficient of Determination (R^2 Score).

Objectives:

1. To collect and preprocess a structured residential dataset capturing core household attributes (occupants, AC usage hours, fan counts, appliances, daily usage hours, ambient temperature, and previous consumption).
2. To perform exploratory data analysis (EDA) and generate visual distribution plots, feature trendlines, and correlation heatmaps illustrating energy dynamics.
3. To design, train, and compare multiple regression models (Linear Regression, Decision Tree Regressor, Random Forest Regressor) using an 80:20 train-test split.
4. To evaluate the candidate models on an unseen test set and select the optimal model based on lowest test Root Mean Squared Error (RMSE) and highest R^2 score.
5. To serialize and persist the best-performing model using Joblib and build a responsive Flask-based web application for real-time inference.
6. To provide real-time daily predictions (kWh/day), calculate estimated monthly consumption (kWh/month), and persist prediction history with timestamps in an SQLite database.

Software / Hardware Required:

- **Operating System:** Windows / Linux / macOS
- **IDE:** Visual Studio Code
- **Language:** Python 3.x
- **Backend Framework:** Flask (REST API & MVC Architecture)

- **Machine Learning:** scikit-learn (LinearRegression, DecisionTreeRegressor, RandomForestRegressor)
- **Data Processing & Visualization:** Pandas, NumPy, Matplotlib, Seaborn
- **Database:** SQLite3 (Local prediction history log)
- **Model Persistence:** Joblib
- **Frontend:** HTML5, CSS3 (Modern Light Theme), JavaScript
- **Dataset:** Household Electricity Consumption Dataset (600 records, 8 attributes)
- **Version Control:** Git

Project Directory Structure:

The project ("HouseholdElectricityPrediction") is organized as a clean Flask backend with dedicated data, model, static visual assets, and template directories, as shown below.

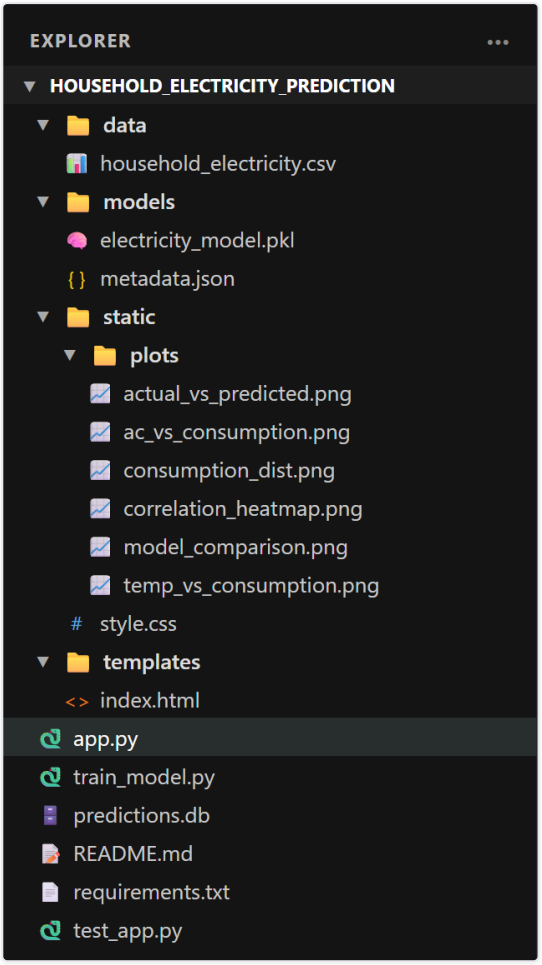


Fig 1: Project directory structure of HouseholdElectricityPrediction in Visual Studio Code

Procedure:

1. Collect and organize a realistic dataset (600 records) capturing 7 input features and the target daily electricity consumption in `data/household_electricity.csv`.
2. Run data preprocessing: verify zero null values, detect and eliminate duplicate rows, and cast columns to appropriate numerical formats (`int64`, `float64`).
3. Execute Exploratory Data Analysis (EDA) to produce Matplotlib/Seaborn visualizations: target consumption histogram, AC vs. kWh trendline, outdoor temperature vs. kWh, previous vs. actual kWh, and the correlation heatmap.
4. Partition the preprocessed dataset into training (80%, 480 samples) and testing (20%, 120 samples) sets using `random_state=42` for strict reproducibility.
5. Train multiple regression algorithms: Ordinary Least Squares (Linear Regression), Decision Tree Regressor, and Random Forest Regressor on the training partition.
6. Evaluate all candidate models on the unseen test set using MAE, MSE, RMSE, and R^2 Score, and select the optimal model with the lowest test RMSE.
7. Serialize the selected best model to `models/electricity_model.pkl` and export metadata to `models/metadata.json` using Joblib.
8. Launch the Flask web application (`app.py`) and access the user interface via local browser at `http://127.0.0.1:5050`.
9. Accept household inputs via the web form with input boundary validation preventing negative or out-of-range inputs.
10. Feed validated inputs to the serialized model, output the predicted daily consumption (kWh/day), calculate the estimated monthly consumption (Daily × 30), and record each query to the SQLite database (`predictions.db`).

Implementation Code:

Representative implementation of the key modules used in the project, based on the pipeline shown in the application (adapt variable/file names to match your exact source files).

1. Data Preprocessing & Pipeline Setup (train_model.py)

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

# Load dataset and verify integrity
df = pd.read_csv('data/household_electricity.csv')
print(f"Dataset shape: {df.shape} | Missing values: {df.isnull().sum().sum()}")

features = ['People', 'AC_Usage_Hours', 'Number_of_Fans', 'Number_of_Appliances',
            'Daily_Usage_Hours', 'Outside_Temperature', 'Previous_Consumption']
target = 'Electricity_Consumption'

X = df[features]
y = df[target]

# 80:20 Train-Test Partition with fixed random seed
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
print(f"Train samples: {len(X_train)} | Test samples: {len(X_test)}")
```

2. Exploratory Data Analysis & Visualization (train_model.py)

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

# 1. Target Consumption Distribution
plt.figure(figsize=(7, 4.5))
sns.histplot(df['Electricity_Consumption'], kde=True, color='#2563eb', bins=25)
plt.title('Distribution of Daily Electricity Consumption', fontweight='bold')
plt.xlabel('Daily Electricity Consumption (kWh/day)')
plt.savefig('static/plots/consumption_dist.png', dpi=150)
plt.close()

# 2. AC Usage vs. Electricity Consumption
plt.figure(figsize=(7, 4.5))
sns.regplot(data=df, x='AC_Usage_Hours', y='Electricity_Consumption',
            scatter_kws={'alpha': 0.5, 'color': '#0284c7'}, line_kws={'color': '#e11d48'})
plt.title('AC Usage Hours vs Electricity Consumption', fontweight='bold')
plt.savefig('static/plots/ac_vs_consumption.png', dpi=150)
plt.close()

# 3. Feature Correlation Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(df.corr(), annot=True, fmt='.2f', cmap='coolwarm', square=True)
plt.title('Feature Correlation Heatmap', fontweight='bold')
plt.tight_layout()
plt.savefig('static/plots/correlation_heatmap.png', dpi=150)
plt.close()
```

3. Model Training, Comparison & Serialization (train_model.py)

```
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import joblib

models = {
    'Linear Regression': LinearRegression(),
    'Decision Tree Regressor': DecisionTreeRegressor(max_depth=6, random_state=42),
    'Random Forest Regressor': RandomForestRegressor(n_estimators=100, max_depth=8, random_state=42)
}

results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    results[name] = {
        'MAE': mean_absolute_error(y_test, preds),
        'RMSE': np.sqrt(mean_squared_error(y_test, preds)),
        'R2': r2_score(y_test, preds)
    }

# Select best model with lowest RMSE
best_model_name = min(results.keys(), key=lambda k: results[k]['RMSE'])
best_model = models[best_model_name]
joblib.dump(best_model, 'models/electricity_model.pkl')
print(f"Selected Best Model: {best_model_name} (Saved to models/electricity_model.pkl)")
```

4. Prediction Route & Estimation Backend (app.py)

```
from flask import Flask, render_template, request, flash
import joblib
import pandas as pd
import numpy as np

app = Flask(__name__)
MODEL_PATH = 'models/electricity_model.pkl'

@app.route('/predict', methods=['POST'])
def predict():
    try:
        # Extract and validate input values
        people = float(request.form.get('people', 0))
        ac_hours = float(request.form.get('ac_usage_hours', 0))
        fans = float(request.form.get('number_of_fans', 0))
        appliances = float(request.form.get('number_of_appliances', 0))
        daily_hours = float(request.form.get('daily_usage_hours', 0))
        temp = float(request.form.get('outside_temperature', 0))
        prev_kwh = float(request.form.get('previous_consumption', 0))

        # Input boundary validation
        if people <= 0 or ac_hours < 0 or daily_hours <= 0:
            flash("Invalid input values. Please provide positive numbers.", "danger")
            return render_template('index.html')

        # Load serialized model
        model = joblib.load(MODEL_PATH)
        feature_names = ['People', 'AC_Usage_Hours', 'Number_of_Fans', 'Number_of_Appliances',
                         'Daily_Usage_Hours', 'Outside_Temperature', 'Previous_Consumption']
        input_df = pd.DataFrame([[people, ac_hours, fans, appliances, daily_hours, temp, prev_kwh]],
                                columns=feature_names)

        # Model inference
        raw_pred = model.predict(input_df)[0]
        predicted_daily = max(0.5, float(raw_pred))
        predicted_daily_rounded = round(predicted_daily, 2)

        # Monthly Consumption Estimate (Daily x 30)
        estimated_monthly_rounded = round(predicted_daily_rounded * 30.0, 2)

        return render_template('index.html',
                               prediction_result={
                                   'daily_kwh': predicted_daily_rounded,
                                   'monthly_kwh': estimated_monthly_rounded
                               })
    except Exception as e:
        flash(f"Error during prediction: {e}", "danger")
        return render_template('index.html')
```

5. Saving Prediction History to SQLite (app.py)

```
import sqlite3
from datetime import datetime

def log_prediction(people, ac_hours, fans, appliances, daily_hours, temp, prev_kwh, daily_pred,
monthly_est):
    conn = sqlite3.connect('predictions.db')
    cursor = conn.cursor()
    cursor.execute('''CREATE TABLE IF NOT EXISTS predictions (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        timestamp TEXT NOT NULL,
        people INTEGER, ac_hours REAL, fans INTEGER, appliances INTEGER,
        daily_hours REAL, temperature REAL, prev_consumption REAL,
        predicted_daily REAL, estimated_monthly REAL
    )''')

    timestamp_str = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    cursor.execute('''INSERT INTO predictions (
        timestamp, people, ac_hours, fans, appliances,
        daily_hours, temperature, prev_consumption,
        predicted_daily, estimated_monthly
    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)''', (
        timestamp_str, int(people), round(ac_hours, 1), int(fans),
        int(appliances), round(daily_hours, 1), round(temp, 1),
        round(prev_kwh, 2), daily_pred, monthly_est
    ))
    conn.commit()
    conn.close()
```

Output (Step-by-Step Screenshots):

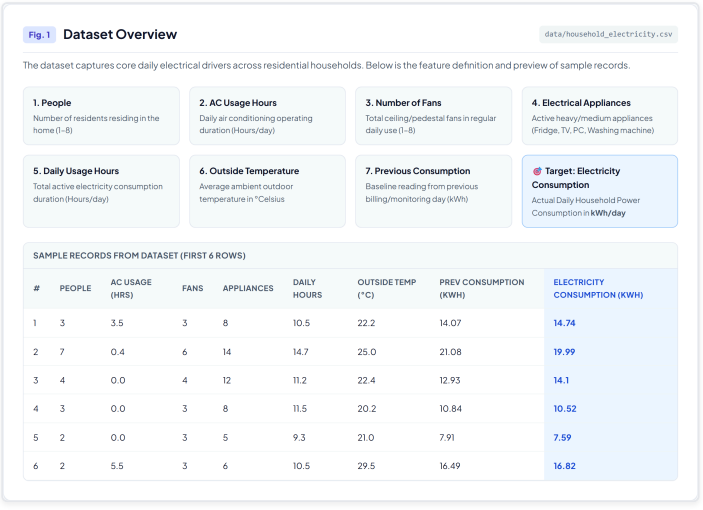


Fig 2: Step 1 — Dataset Overview: dataset size (600 records), 7 input features, and sample records preview

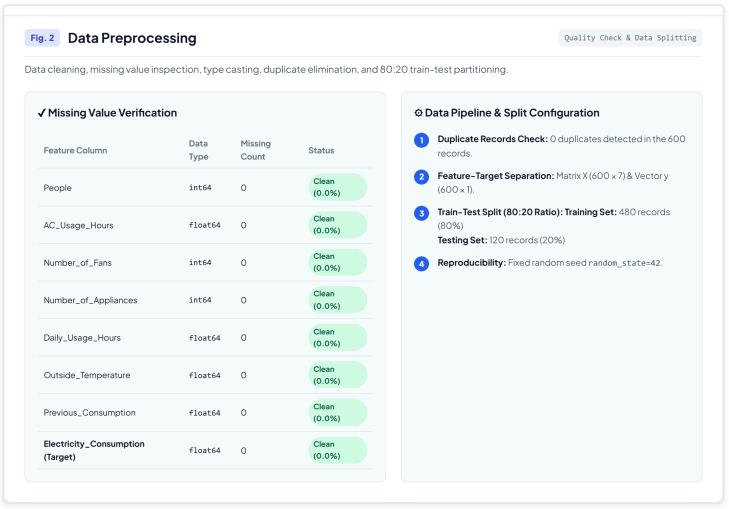


Fig 3: Step 2 — Data Preprocessing: missing-value verification, data types, and 80:20 train-test split configuration

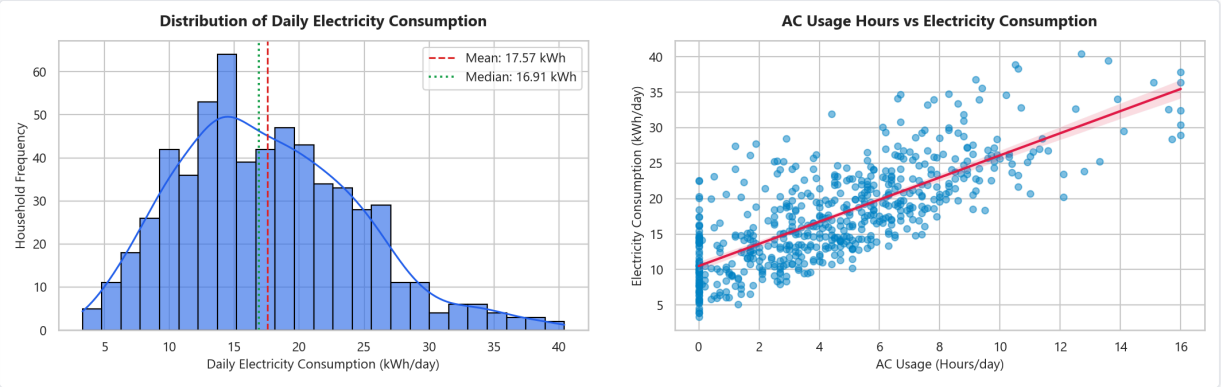


Fig 4: Step 3 — Exploratory Data Analysis: daily electricity consumption distribution and AC usage hours impact

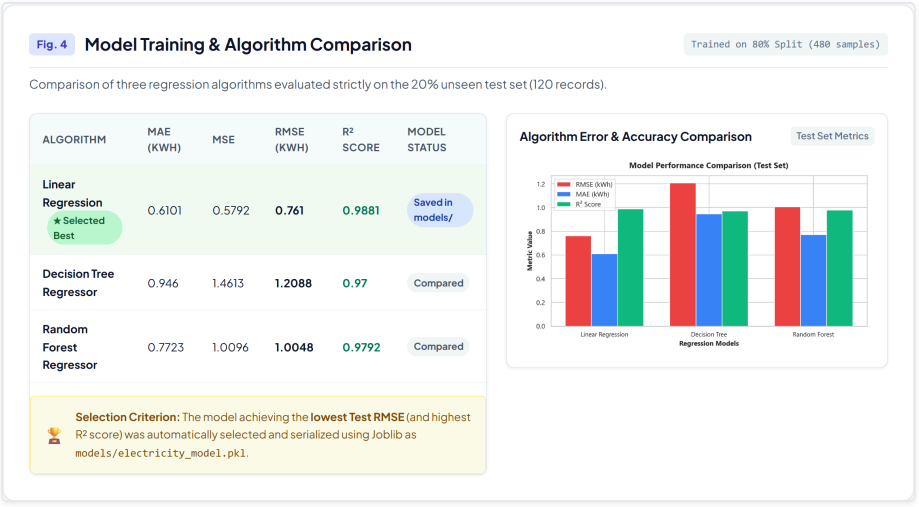


Fig 5: Step 4 — Model Training & Comparison: Linear Regression vs. Decision Tree vs. Random Forest Regressor on unseen test set

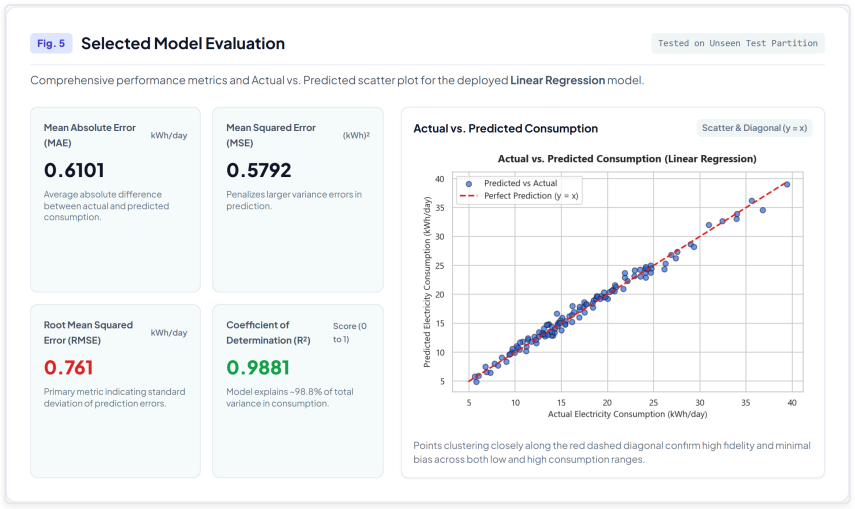


Fig 6: Step 5 — Selected Model Evaluation: MAE (0.6101), MSE (0.5792), RMSE (0.7610), R² (0.9881), and Actual vs. Predicted scatter plot

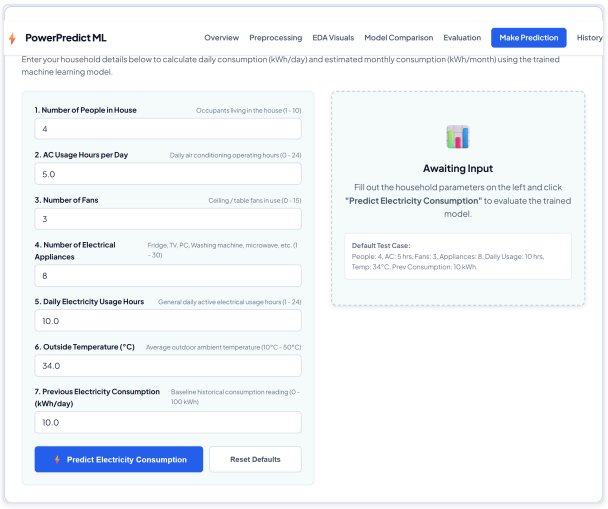


Fig 7: Step 6 — Electricity Consumption Prediction: interactive input form with 7 household and environmental features

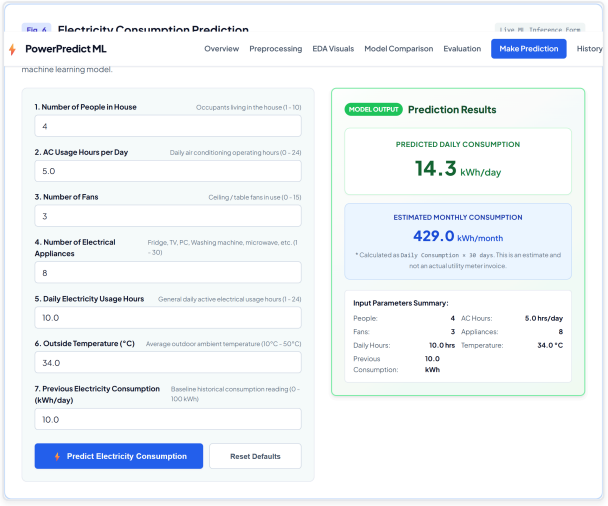


Fig 8: Step 6 — Prediction output (Test Case 1): Predicted Daily Consumption (14.30 kWh/day) and Estimated Monthly Consumption (429.00 kWh/month)

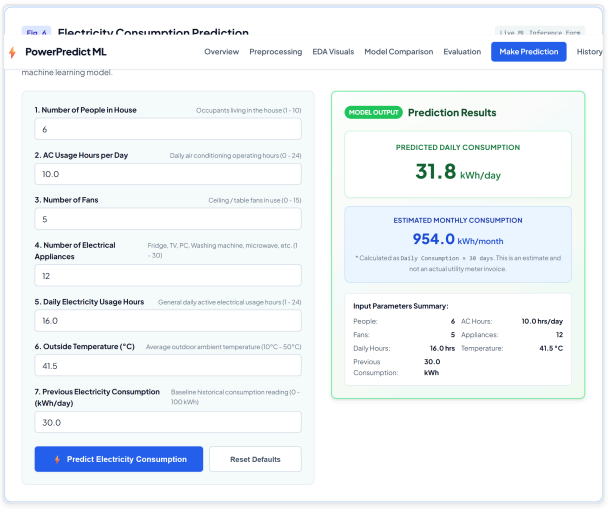


Fig 9: Step 6 — Prediction output (Test Case 2 - High Summer Demand): Predicted Daily Consumption (31.80 kWh/day) and Monthly Estimate (954.00 kWh/month)

Log **Recent Prediction History**

SQLite3: predictions.db Clear History

Local database log of user queries and predicted consumption values generated during this session.

TIMESTAMP	PEOPLE	AC (HRS)	FANS	APPLIANCES	DAILY (HRS)	TEMP (°C)	PREV (KWH)	PREDICTED DAILY (KWH)	ESTIMATED MONTHLY (KWH)
2026-10-20 18:17:31	6	10.0	5	12	16.0	41.5	30.0	**31.8 kWh**	954.0 kWh
2026-10-20 18:17:28	4	5.0	3	8	10.0	34.0	10.0	**14.3 kWh**	429.0 kWh
2026-10-04 20:23:05	4	5.0	3	8	10.0	34.0	10.0	**14.3 kWh**	429.0 kWh

Fig 10: Step 7 — Prediction history log persisted in the SQLite database (predictions.db)

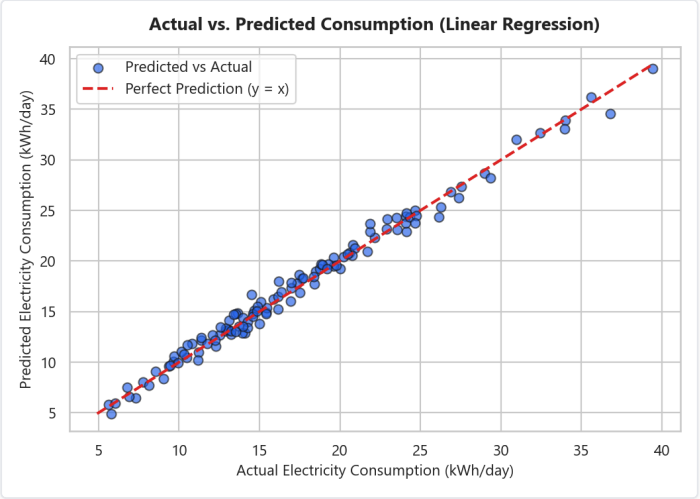


Fig 11: Step 8 — Actual vs. Predicted Electricity Consumption scatter plot with ideal identity diagonal (y = x)

Result:

A web-based Household Electricity Consumption Prediction system was successfully designed, trained, evaluated, and implemented using a Flask web backend, Scikit-learn regression algorithms, and an SQLite database. The dataset comprising 600 records across 7 core electrical and environmental features was preprocessed, verified for zero missing values and zero duplicate records, and partitioned using an 80:20 train-test split (random_state=42).

Among the evaluated regression algorithms, Linear Regression demonstrated superior generalization on the unseen test set, achieving a Root Mean Squared Error (RMSE) of 0.7610 kWh/day, a Mean Absolute Error (MAE) of 0.6101 kWh/day, a Mean Squared Error (MSE) of 0.5792 $(kWh)^2$, and a Coefficient of Determination (R^2) of 0.9881 (explaining 98.81% of total consumption variance). The model was serialized using Joblib and deployed within a modern, responsive web application. The web interface enables users to submit household parameters, produces real-time daily consumption predictions along with estimated monthly totals, and logs every query into an SQLite database for auditing and longitudinal energy management.